

Ejercicio 1

a) Errores de sintaxis

- `a = obtenerA()` y `b = obtenerB()`: las variables `a` y `b` no están declaradas. En Java hay que indicar el tipo (por ejemplo `int a = ...`). Además falta el `;` al final de cada línea.
- `if(a = b)`: se usó el operador de asignación `=` en lugar del de comparación `==`. Con `=` se le asigna el valor de `b` a `a`, y la condición del `if` no es un booleano (salvo que ambas sean boolean).
- `System.out.print(a y b son iguales)`: el texto no está entre comillas, así que Java no lo reconoce como un `String`. También falta el `;`.

b) Errores de sintaxis

- `int arreglo` en el parámetro: se usa `arreglo.length` y se accede con `arreglo[i]`, así que debe ser un arreglo (`int[] arreglo`). Un `int` no tiene `length` ni índices.
- `suma = 0`: falta declarar el tipo (`int suma = 0`) y falta el `;`.
- `for( i = 0, i < arreglo.length, i++)`: las partes del `for` se separan con `;` y no con `,`. Además la variable `i` no está declarada (`int i = 0`).
- `suma += arreglo[i]`: está mal escrito el operador. Se interpreta como `suma = +arreglo[i]`, o sea una asignación y no una acumulación. Debe ser `suma += arreglo[i]`. También falta el `;`.
- `return suma/arreglo.length`: falta el `;`. Además el método está declarado `void`, y un método `void` no puede devolver un valor. Debería declararse con un tipo de retorno (`int` o `double`).

c) Falsa

El operador `==` compara referencias, es decir si ambas variables apuntan al mismo objeto en memoria. `equals()` compara el contenido de los `Strings`, o sea si tienen los mismos caracteres. Por ejemplo, `new String("hola") == new String("hola")` da `false`, pero `equals()` da `true`. A veces `==` da `true` con literales, porque Java reutiliza `Strings` iguales en el pool de constantes. Pero no se puede confiar en eso, y para comparar contenido siempre hay que usar `equals()`.

d) Verdadera

Cada llamada recursiva ocupa espacio en la pila de ejecución (`stack`). Si la recursión no termina nunca, la pila se llena y se produce `StackOverflowError`. Esto pasa si:

- no hay caso base,
- el caso recursivo no se acerca al caso base (por ejemplo, llama con los mismos parámetros),
- o la recursión es correcta pero demasiado profunda (por ejemplo, con un n enorme).

Ejercicio 3

a) Enunciados que pueden ser parte del IREP de Libro

I) No es parte del IREP. `numerosInventario` es un arreglo de `int`, un tipo primitivo que nunca puede valer `null`. Esa condición se cumple siempre, así que no restringe nada del estado. Lo que sí podría ser IREP es una condición sobre el valor de los números, por ejemplo que sean de 5 cifras.

II) No es parte del IREP. Describe cómo debe comportarse una operación (el préstamo), no una propiedad que el objeto deba cumplir en todo momento. Eso pertenece a la especificación del método, no al invariante.

III) Sí es parte del IREP. Es una condición sobre el estado interno que debe valer siempre. Los dos arreglos son paralelos, así que la posición `i` de uno corresponde al ejemplar `i` del otro.

b) Invariante de representación (IREP)

Clase Libro:

1. `titulo`, `autor`, `sucursal`, `estante`, `numerosInventario` y `prestadoA` no pueden ser `null`.
2. `titulo`, `autor` y `sucursal` no pueden ser cadenas vacías.
3. `estante` tiene exactamente 8 caracteres.
4. `numerosInventario` y `prestadoA` tienen la misma longitud.
5. Cada elemento de `numerosInventario` es un número de 5 cifras (entre 10000 y 99999).
6. No hay números repetidos en `numerosInventario`, porque son únicos para cada ejemplar del mismo libro.
7. Las posiciones de `prestadoA` pueden ser `null` (ejemplar disponible). Si no lo son, deben referenciar a un `Socio` válido que cumpla su propio IREP, y además activo.

Clase Socio:

1. `nombre` no es `null` ni vacío.
2. `nroSocio` no es `null` y tiene exactamente 10 caracteres.
3. Si `activo` es `true`, entonces `fechaBaja` es `null`.
4. Si `activo` es `false`, entonces `fechaBaja` no es `null` y es una fecha válida.